

Raw Python vs. LangChain Agent

Overview

The LangChain agent developed in this project performs tool-based tasks, maintains conversation memory, handles tool failures, and produces structured final responses. Compared with the raw-Python agent developed on Day 1, LangChain provides higher-level abstractions that significantly reduce the amount of code required to manage the agent workflow.

Raw Python vs. LangChain

In the raw-Python implementation, the agent workflow had to be managed manually. This included constructing API requests, defining available tools and their schemas, sending tool calls to the model, processing function-call responses, executing the requested Python functions, adding tool results back into the conversation, and managing the message history. The developer therefore had direct control over the complete agent loop, but this also required more code and more careful handling of API-specific details.

LangChain abstracts much of this orchestration. Tools can be registered using the `@tool` decorator, `AgentExecutor` manages the agent execution loop, conversation memory can be attached to the executor, and `with_structured_output()` can enforce a predefined Pydantic response schema. This made the implementation shorter, more modular, and easier to extend. Adding a new tool or changing the response structure required considerably less low-level message management than in the raw-Python implementation.

The main trade-off is abstraction leakiness. LangChain hides much of the underlying agent loop, so debugging sometimes requires inspecting the verbose execution trace to understand which tool was selected, what result it returned, and how the agent proceeded. Error handling is also not completely automatic: the unreliable product lookup required explicit `try/except` handling so that a failure could be returned to the agent as an observable `TOOL_ERROR` instead of terminating execution.

Annotated Execution Trace

The following trace demonstrates how the LangChain agent recovered from a tool failure:

- 1. Reason:** The agent determines that the product lookup tool should be used to find the Keyboard price.
- 2. Act:** `unreliable_product_lookup("Keyboard")` is invoked.
- 3. Observe:** The tool returns `TOOL_ERROR: Product database temporarily unavailable`. The exception is contained inside the tool rather than crashing the agent.
- 4. Reason:** Because the requested lookup failed, the agent uses the alternative CSV data source specified in the user's request.
- 5. Act:** `read_csv("products.csv")` is invoked.

6. Observe: The CSV contains Keyboard = 4,500, along with the other product prices.

7. Final Answer: The agent explains the temporary lookup failure and provides the Keyboard price obtained from the CSV.

Conclusion

Overall, LangChain made tool registration, orchestration, memory, error recovery, and structured output substantially easier than the raw-Python approach. The raw implementation provided greater visibility and control over the underlying API interactions, while LangChain provided a cleaner and more maintainable abstraction for building an agent. The main limitation I observed was that some of the framework's behavior is hidden behind abstractions, meaning that understanding execution traces is important when debugging or handling unexpected behavior.